

awesome-gpt-image-2 小白指南

Prompt as Code: 把"抽卡的玄学"变成"可参数化的代码资产" —— 541 条逆向工程提示词案例库全解

🏆 GitHub 趋势榜 第一

📁 541 条结构化案例

✂️ 21 套工业模板

🤖 自带 Agent Skill

上游仓库: github.com/freestylefly/awesome-gpt-image-2 (作者: 苍何) | 学习产出:
F:\reminder\awesome_gpt_image_2\ | 2026-10-02

? 疑问即目录 (工科毕业生十二连问)

Q1 提示词合集凭什么登顶趋势榜?

Q2 Prompt as Code 是什么?

Q3 "逆向工程"具体怎么做的?

Q4 541 个案例怎么组织?

Q5 合格提示词长什么样 (六块协议)?

Q6 散文式 vs 结构化差在哪?

Q7 [占位符] 是什么套路?

Q8 工业模板+避坑指南说了什么?

Q9 541 条数据里有什么规律? (实测)

Q10 自带的 Agent Skill 干嘛的?

Q11 我能拿它干什么? (能力清单)

Q12 怎么保证我写的提示词合格?

Q1 一个"提示词合集"仓库, 凭什么登顶 GitHub 趋势榜第一?

GPT-Image2 火了之后, X (推特) 上每天都有人晒"神图"并附上提示词。苍何干的事情是: 把这些散落的提示词一条条扒下来、**逆向拆解、去掉原文照搬 (全部 AI 重写保证原创)、打上类别/风格/场景标签**, 攒成了一个 541 条案例的数据库——最初是 329 条, 如今 541 条。推文说的就是它先冲到趋势榜第二 (仅次于 Codex), 随后登顶第一。

登顶的原因不是"收集得多", 而是它把散装经验做成了**三件别人没有的东西**: ① **结构化数据** (cases.json, 可以被脚本和 Agent 检索); ② **工业模板** (从案例中提炼的 21 套"填空即用"模板, 每套带避坑指南); ③ **Agent Skill** (把整个库打包成 AI 编程助手能调用的技能包)。



图1: 登顶之路 = 把散装经验重造成"可复用的数据资产"

Q2 "Prompt as Code (提示词即代码)" 到底是什么意思?

一句话定义：把图像提示词从"一次性的文案"升级为**有固定结构、有参数槽位、可批量生成、可测试质检的代码资产**。仓库原话：把散文式提示词**压缩成结构化协议** (compress prose-style prompts into structured protocols)。

我们知识库里有 **[[提示词模板]]** (LangChain 的带变量槽位提示词对象)。Prompt as Code 是同一思想在**图像生成**领域的落地，但更进一步：文本模板管"话术里的槽位"，Prompt as Code 管的是**整条提示词的"架构"——主体、构图、风格、文字、比例、约束各占一个固定的"代码块"**，像写代码一样分模块，像改常量一样改参数。

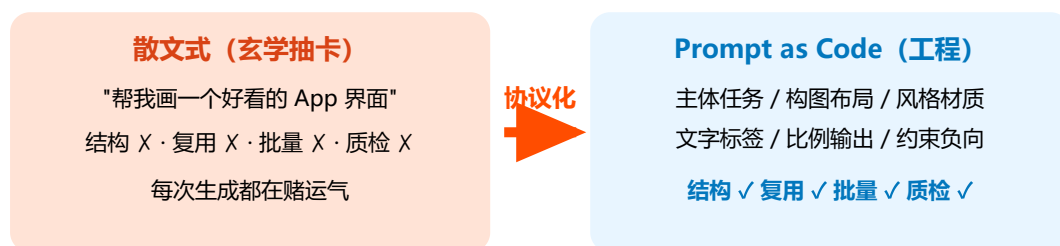


图2：同一个需求的两种写法——右边的"每个格子"都可控、可替换、可检查

类比：散文式提示词像"跟装修师傅口头说'给我装得温馨点'"; Prompt as Code 像**施工图纸**——哪面墙什么颜色、插座打在哪、用什么牌子的漆，全部写死。师傅（模型）没了自由发挥，你也没了惊吓。

Q3 "逆向工程提示词"具体是怎么做的？

仓库自嘲是"无情逆向机器"。流程其实很朴素，分四步：

步骤	做什么	本仓库的对应物
① 收集	蹲守 X 上带提示词的 GPT-Image2 晒图帖，记录图 + 原提示词 + 作者	每条案例都有 <code>sourceUrl</code> 指回原推
② 拆解	按六块协议把散文提示词归位（见 Q5），提炼出可复用的"骨架"	docs/templates.md 的 21 套模板
③ 重写	用 AI 把原文重写（避免照搬侵权），保留结构	README 标注 100% Original AI Rewritten
④ 归档	打标签（类别/风格/场景）、编 case id、生成画廊和 JSON 数据	cases.json + docs/gallery-*.md



图3：逆向四步流水线——本质是"数据工程"，不是玄学

关键洞察：逆向的对象不是模型权重，而是**社区验证过的提示词写法**。"这张图为什么好"的答案，往往就藏在它提示词的某个结构块里。

Q4 541 个案例是怎么组织的？——三层标签体系 + 一份 JSON 数据库

每条案例是一个 JSON 对象：`id / title / prompt / category / styles / scenes / sourceUrl / githubUrl`。
组织检索靠三层标签：

- **类别 category (13 个)**：回答"画的是什么类型"——海报字体 90、摄影写实 78、UI 界面 73、插画艺术 59、图表信息图 53、产品电商 42.....
- **风格 styles (19 个)**：回答"什么质感"——UI(250)、Realistic(217)、Poster(199)、3D、Classical.....
- **场景 scenes (10 个)**：回答"用在什么场合"——Tech、Commerce、Education.....

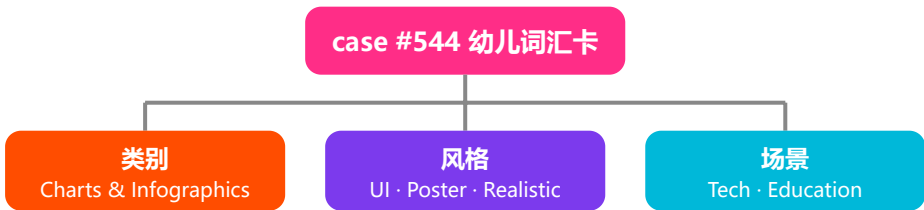


图4：一条案例的三层标签——Agent 可以按"类别+风格+场景"组合检索

这份 `data/cases.json` 就是整个项目的"心脏"：网站画廊、PDF 文档、Agent Skill 全部从它生成（`npm run generate:style-skill`）。**数据和展示分离——这是典型的工程做法。**

Q5 一条合格的图像提示词长什么样？——六块协议（本指南最重要一页）

拿仓库案例 #544 《幼儿词汇拆解学习卡》开刀，任何好提示词都能拆进这六个抽屉：

块	#544 原文怎么写	它锁定了什么
① 主体任务	Create a clean, child-friendly educational vocabulary poster...	画什么 + 给谁用，一句话锁边界
② 构图布局	[FRUIT] on the left ... [PART] on the right. Connect with a dotted arrow...	左大右小 + 箭头连接，空间关系写死
③ 文字标签	the word "[FRUIT NAME]" in large bold uppercase letters at the top...	文字逐字指定——文字渲染是图像模型最大翻车点
④ 风格材质	pastel-blue background, rounded panels, realistic fruit photography...	风格靠具体名词堆，不靠"好看"这种虚词
⑤ 整体氛围	should feel bright, educational, modern, uncluttered...	兜底氛围句，补前四块管不到的整体感
⑥ 比例输出负向	Vertical 4:5 composition, high resolution, ... no unnecessary decorations	比例 + 分辨率 + "不要什么"收尾

讲个故事：把提示词想象成给一位外国天才画师下订单。他画技无敌但完全不懂"潜台词"——你说"好看"，他就按自己审美来；你写"左 2/3 放苹果实物、右侧放横切面、虚线箭头连接、顶部大写粗体标苹果英文"，他就分毫不差。六块协议，就是把"订单"说成他无法误解的程度。



六块顺序可以变，但缺块就会“抽卡”

图5：六块协议——awesome-gpt-image-2 全部 21 套模板的共同骨架

Q6 散文式 vs 结构化，差别到底在哪？（同一个需求两种写法）

```
// 散文式（抽卡）
画一个好看的健身 App 界面

// Prompt as Code（结构化，来自仓库 UI 模板）
为[产品类型]生成一张[平台]界面图。
核心功能：[功能点A]、[功能点B]、[功能点C]。
视觉风格：[极简/科技/拟物]，主色[颜色]，强调色[颜色]。
布局：[顶部导航/双栏/卡片流]，信息层级清晰，留白充足。
输出：高保真UI截图，文字清晰可读，比例[9:16/16:9]。
```

维度	散文式	结构化
可复现	同一个提示词两次出图天差地别	关键变量全锁定，波动只在细节
可批量	改需求=重写整段文案	改一个参数即可批量生产变体
可质检	没法机器检查“写得全不全”	缺哪一块一查便知（见实验4 Lint）
可给 Agent 用	模糊指令让 Agent 无所适从	JSON 模板可直接作为 API 入参

仓库还提供 **JSON 进阶模板**（"推荐给 Agent 调用"）：把六块写成 JSON 字段（`platform / layout / style.primary_color / content.cards...`），Agent 填 JSON 就像填表单，生成前还能校验字段合法性。

Q7 提示词里的 [FRUIT] 占位符是什么套路？

逆向出来的社区提示词里大量出现 [FRUIT]、[PART NAME]、[产品类型] 这类**大写方括号槽位**。它是提示词界的"变量名"：原帖作者晒的是"苹果版"，你把 [FRUIT] 换成"芒果"，就得到同款构图的芒果版——**一张图的方法论，变成一类图的生成器**。

和知识库里 **[[提示词模板]]**（LangChain 的 {variable} 语法）对比：思想一模一样，只是语法不同、运行时不同——图像提示词的"运行时"是**你自己（或脚本）做字符串替换**，LangChain 的运行是框架的 `format()`。实验 3 就是用 Python 函数实现了这个替换。

防坑（实测）：占位符替换完必须检查没有残留——实验 4 的 Lint 里专门有一条"无残留占位符"，因为把 [FRUIT] 原样发给模型，模型真可能画一个写着"FRUIT"的水果。

Q8 21 套工业模板 + 避坑指南，都讲了什么？

`docs/templates.md` (1080 行) 从案例里提炼了 21 套模板，覆盖 13 个类别，每套 = 常规模板（填空散文） + JSON 进阶模板（给 Agent） + 避坑指南。避坑指南全是实操血泪，摘两套感受一下：

模板	避坑指南（原文摘录）
UI 与界面	不要给模糊指令，明确"平台+比例+布局"，否则模型像实习生乱排版； 强制文字锁定 ，要求"文字绝对可读"，避免乱码按钮；截图要区分平台特征（X 有蓝勾认证、小红书是双列瀑布流）；车机等特殊屏幕比例（21:9）必须写在最前面。
图表与信息图	主题要具体："老年人日常健康管理指南"而非"健康"；模块 3-5 个并写明模块间逻辑（箭头/连线）；先定图表类型（流程图/对比图/时间线）再填内容。

模板 = 骨架（固定话术） + [槽位]（每张图变化的部分） + 避坑指南（前人踩坑清单）
"直接复制填空，再也不用玄学抽卡" —— 仓库原话

图6：模板三件套。21 套模板 = 21 个开箱即用的"图纸"

Q9 541 条数据里藏着什么规律？（实验 1 实测）

我们把 `cases.json` 当数据库查了一遍（脚本见 `exercise\ex1_stats.py`，输出为真实运行结果）：

指标	实测值	解读
案例总数	541（13 类别 / 19 风格 / 10 场景）	README 写 544，数据文件 541——以数据为准
类别 Top3	海报字体 90 · 摄影写实 78 · UI 73	"能当生产资料用"的类型最多
提示词长度	平均 1257 字符（中位 1041，最长 8143）	好提示词约 200-300 个汉字，一句话远远不够
风格/材质描述覆盖	73%	最普及的结构块
文字渲染要求覆盖	64%	印证"文字是重点关照对象"
比例锁定覆盖	27%（负向约束 28%、占位符 5%）	社区提示词也不全规范——所以需要模板和 Lint 补齐

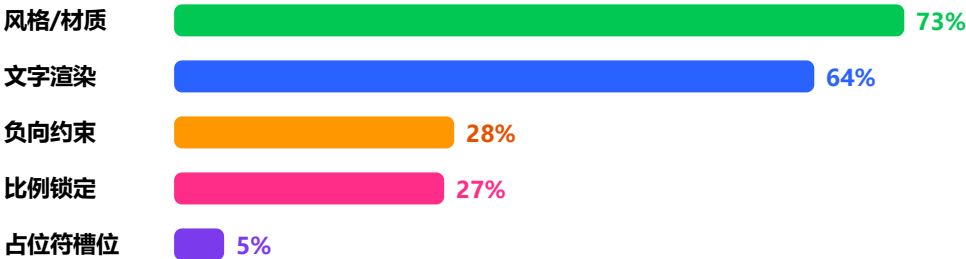


图7：541 条案例的结构化特征覆盖率（ex1_stats.py 实测绘制）

Q10 仓库自带的 Agent Skill 是干嘛的？

`agents/skills/gpt-image-2-style-library/` 是一个标准的 `SKILL.md 技能包`（同知识库 `[[AgentSkills技能包]]` 的格式），还发布成了 npm 包 `gpt-image-2-style-library`。它教会 AI 编程助手：

- 1. 听懂意图 → 按序匹配：模板类别 → 风格标签 → 场景标签 → 最近的案例；
- 2. 候选不止一个时，给 2-3 个选项让用户挑，不自作主张；
- 3. 用六块结构组装最终提示词（主体任务/构图布局/风格材质/文字标签/比例输出/约束负向）；
- 4. 输出默认"可复制的提示词放最前"，约束写具体（精确文字、比例、可读标签）。

妙处：Skill 的知识源就是 `data/style-library.json`——和网站、文档同源生成。改一处数据，网站、Skill、文档 三处同步。这就是"数据即单一事实源（single source of truth）"。

Q11 能力清单：我能拿它干什么？

能力	怎么用（结合我们已有水平）
 查模板直接抄	打开 <code>docs/templates.md</code> ，按类别找模板填空，配任何图像模型（GPT-Image、即梦、可画、GLM 出图）都能用—— 模板是模型无关的方法论
 检索同款案例	在 <code>cases.json</code> 里按类别+风格筛，看 541 条真实提示词怎么写（每条带原图对比）
 装进 AI 编程助手	<code>npm run install:skill</code> 把 style-library 装进 Claude Code / Codex 的技能目录，以后说“帮我生成 XX 图的提示词”自动走模板
 批量生产变体	像实验 3 那样把模板写成函数，一个循环生成 100 张商品图的提示词
 逆向自己的好图	按 Q3 四步流水线，把自己满意的出图提示词拆解归档，攒私有库
 学提示词工程	21 套模板的避坑指南本身就是最好的提示词工程教材

Q12 怎么保证自己拼出来的提示词是合格的？——Prompt Lint

模板是“写对”的方法，还需要“查错”的机制。我们把避坑指南翻译成 **7 条确定性规则**（主体明确/比例锁定/文字要求/风格材质/构图布局/负向约束/无残留占位符），写成了 `ex4_lint.py`——一个提示词“出厂质检器”。这正是知识库 **[[质检Gate与自我纠错循环]]** 在提示工程里的落地：**裁判必须是确定性脚本**，LLM 拿到“哪条不达标”的具体清单才能自动改写重提，形成纠错循环。



图8：组装 → Lint → 过 Gate 才出场的提示词流水线



动手实验：解构 → 重组 → Lint 流水线（全部真实运行）

环境：Windows 11 + Python 3 (`setx PYTHONUTF8 1` 已设，防中文 GBK 假失败)。材料在 `F:\reminder\awesome_gpt_image_2\exercise\`。四步实验都不调外部 API，纯本地跑，随时可复现。

实验 1 | 把 541 条案例当数据库查 (`ex1_stats.py`)

操作： `cd F:\reminder\awesome_gpt_image_2\exercise && python -X utf8 ex1_stats.py` 目的：验证"这是一个结构化数据集，不是句子合集"。

案例总数：541 (13 类别 / 19 风格 / 10 场景)
类别Top3：海报字体 90 · 摄影写实 78 · UI 73
提示词长度：最短 10 / 平均 1257 / 中位 1041 / 最长 8143
结构化覆盖率：风格材质 73% · 文字渲染 64% · 比例锁定 27% · 占位符 5%

✅ 预期结果：终端按行打印类别分布条形图 (# 号) 与覆盖率百分比；同时发现数据文件是 541 条 (README 写 544)，学会"以数据为准"。

实验 2 | 解剖一条真实提示词 (`ex2_deconstruct.py`)

操作： `python -X utf8 ex2_deconstruct.py` 目的：把案例 #544 的原文亲手拆进六块协议。

✅ 预期结果：脚本先打印 #544 英文原文全文，再逐块输出「①主体→⑥输出」六段：每段给原文节选 + 一句"它锁定了什么"。拆完你就掌握了逆向的核心动作。

实验 3 | Prompt as Code: 模板函数化组装 (`ex3_assemble.py`)

操作： `python -X utf8 ex3_assemble.py` 目的：体验"改一个参数=换一版提示词"。

【版本A】...主色荧光绿... 【版本B】...主色落日橙...
【两版 diff】
-视觉风格：极简，主色荧光绿，强调色深灰。
+视觉风格：极简，主色落日橙，强调色深灰。

✅ 预期结果：两版提示词并排打印 + unified diff 只有 1 行不同——结构完整性与参数化同时得到验证。

实验 4 | Prompt Lint 出厂质检 (ex4_lint.py)

操作: `python -X utf8 ex4_lint.py` 目的: 用确定性规则拦截不合格提示词。

[合格样例 (实验3组装结果)] 得分 100/100  PASS

[不合格样例 "画一个好看的app界面"] 得分 14/100  FAIL

X 比例锁定 / X 文字渲染 / X 风格材质 / X 构图布局 / X 负向约束 ...

 **预期结果:** PASS/FAIL 判定 + 逐条失败原因清单。把 FAIL 清单喂回给 LLM 改写, 就是一条完整的自纠错循环。

🌟 一页带走

① awesome-gpt-image-2 的本质是把社区提示词经验重造成数据资产 (541 案例 + 21 模板 + 1 个 Skill), 所以能登顶趋势榜; ② 核心方法论是 **Prompt as Code**: 六块协议 (主体/构图/文字/风格/氛围/输出负向) + [槽位] + 避坑指南; ③ 好提示词平均 1257 字符——一句话抽不出好图; ④ 先查模板再填空, 提交前跑一遍 **Prompt Lint**; ⑤ 模板是模型无关的, GPT-Image、即梦、GLM 出图都能套用。

F:\reminder\awesome_gpt_image_2 · 学习产出 PDF · 2026-10-02 · 配套实验脚本 exercise\ex1~ex4 · 知识入库: vault\概念\与 vault\项目笔记\awesome_gpt_image_2.md